ShelfWise

A campus library's lending desk, carried from a laptop to a monitored Kubernetes cluster

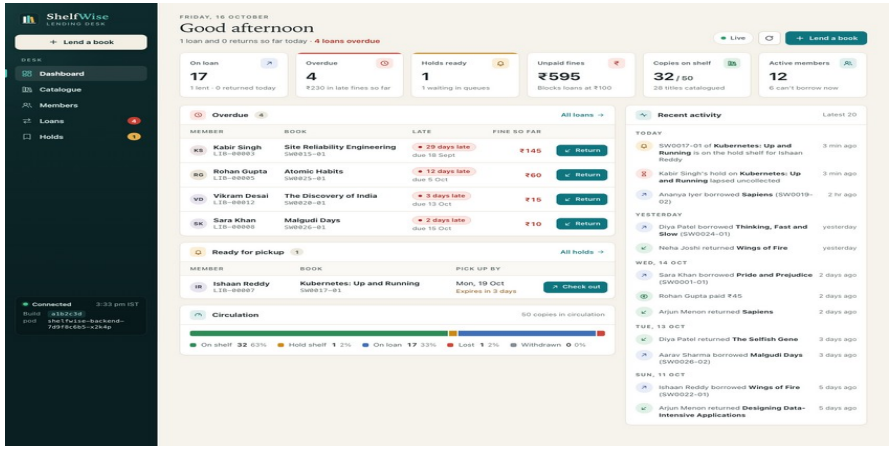
Shubham Kumar · 24BCS10320 · DevOps Heros, Session 21 final capstone · October 2026

74 tests | 10 CI jobs | Trivy-gated images | GitOps with Argo CD | AWS VPC + EKS

What ShelfWise does

Librarians catalogue books and their physical copies, register students and faculty, lend and take back copies, and run the holds queue for popular titles.

Every refusal is a rule, not a crash: the API answers 409 with a sentence the desk can read out, and the rules are published at /api/info so the UI never hardcodes them.



One copy, one borrower

Row locks plus a partial unique index: two API replicas can never lend the same copy.

Loan limits by member

Students 3 books for 14 days, faculty 6 for 28. Two renewals, never while someone is waiting.

Fines that matter

₹5 per day late, capped at ₹200. ₹100 unpaid (or any overdue book) blocks new loans.

A fair holds queue

A returned copy goes to the first member waiting; they have 3 days to collect it.

Tech stack

APPLICATION

React 19 + Vite 8 · FastAPI 0.143 · SQLAlchemy 2.1 · Alembic 1.20

DATA

PostgreSQL 17 · CHECK constraints · partial unique indexes

QUALITY

pytest 9 · ruff · 60-day simulation · multi-process races

CONTAINERS

Multi-stage Docker · nginx-unprivileged · compose

CI/CD

GitHub Actions (10 jobs) · GHCR · SHA-tagged images

SECURITY

Trivy 0.76 · Gitleaks 8.30 · Bandit · pip-audit · npm audit

INFRASTRUCTURE

Terraform 1.15 · AWS provider 6.69 · VPC + EKS 1.35

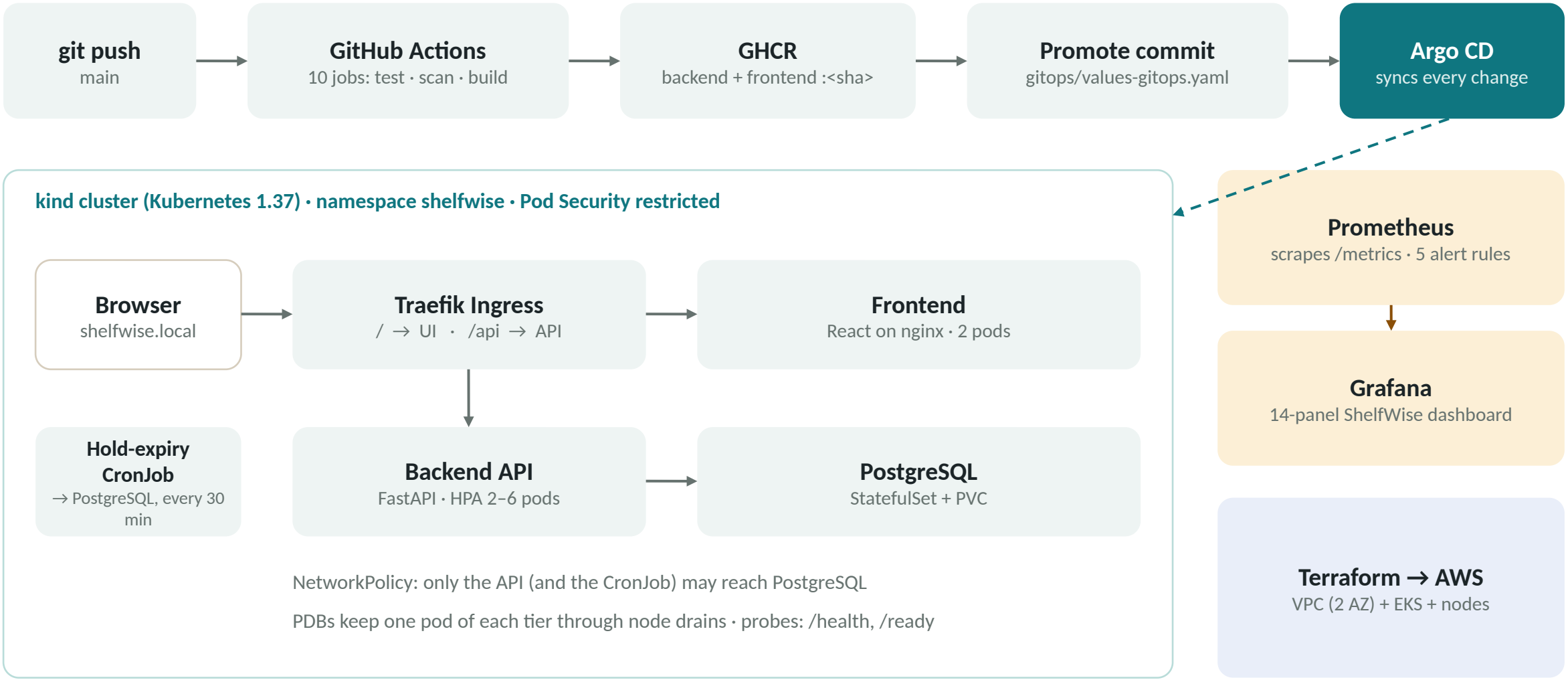
KUBERNETES

kind 1.37 · Helm 4 · Traefik 3.7 · HPA · PDB · NetworkPolicy

GITOPS + OBSERVABILITY

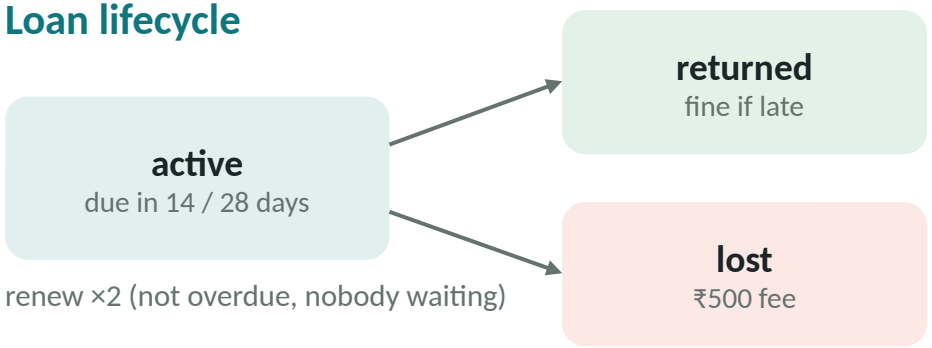
Argo CD 3.5 · Prometheus · Grafana · Alertmanager

Architecture: from git push to a monitored cluster



Rules the database enforces too

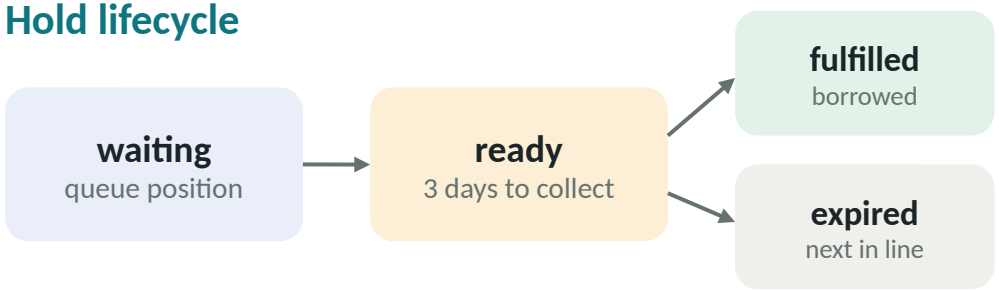
Loan lifecycle



renew ×2 (not overdue, nobody waiting)

Any move out of returned or lost → 409 loan_closed

Hold lifecycle



copy returned → first waiting member · cancel at any open stage

If a bug slipped past the API, PostgreSQL still says no

Partial unique index

one active loan per copy; one open hold per member and book; one ready hold per copy

CHECK constraints

renewals 0–2, due after borrowed, fines ≥ 0, closed_at set exactly when closed, valid status values

Row locks, fixed order

member → book → its loans/holds, so concurrent desks queue instead of deadlocking

Advisory lock on migrate

every replica runs alembic upgrade at start; one migrates, the rest wait

Testing: rules, migrations and real races

74

tests pass on PostgreSQL in CI (72 + 2 skipped on SQLite locally)

93%

line coverage of the backend

1 / 10

replicas racing for the last copy get it; 9 get a clear 409

3 / 6

loans for one student at six desks at once (limit 3)

How the tests stay honest

- Never the real database: a throwaway SQLite file, or a disposable PostgreSQL in CI
- Schema built by running the real Alembic migrations (up, down, up)
- A 60-day simulated library checks invariants afterwards
- Races run as separate processes: threads were staggered by the GIL and proved nothing

Mutation test: remove the row locks

One student, six desks: all 6 loans go through — the limit of 3 is broken.

Last copy, ten desks: the unique index still stops a double loan, but 9 members get a vague "retry" instead of "no copy on the shelf".

Both race tests fail, every time — so they really test the locks.

CI/CD: every push to main, gate by gate

Gates in parallel

- pytest + ruff
- PostgreSQL: migrations, lock, races
- Frontend build + npm audit
- Gitleaks · Bandit · pip-audit
- Helm lint + kubeconform
- Terraform validate + Trivy IaC

→ **Build + Trivy**
both images; fixable HIGH/CRITICAL fail

→ **Push to GHCR**
:<commit sha>, the scanned tarball

→ **Deploy in kind**
helm upgrade --install + smoke test

→ **Promote**
commit tag to values-gitops.yaml

3m 35s

for a full green run (run #10, the live demo push)

SHA

image tags are full commit SHAs; nothing is ever tagged latest

0

deploy credentials in CI: Argo CD pulls from Git

DevSecOps: seven automated checks

Gitleaks
secrets in the whole git history

Bandit
Python SAST, medium+

pip-audit + npm audit
known-vulnerable dependencies

Trivy image ×2
OS + library CVEs, secrets in layers

Trivy config
Terraform misconfigurations

Pod Security restricted
non-root, no caps, seccomp, RO root

Pinned supply chain
actions by commit SHA, images by digest

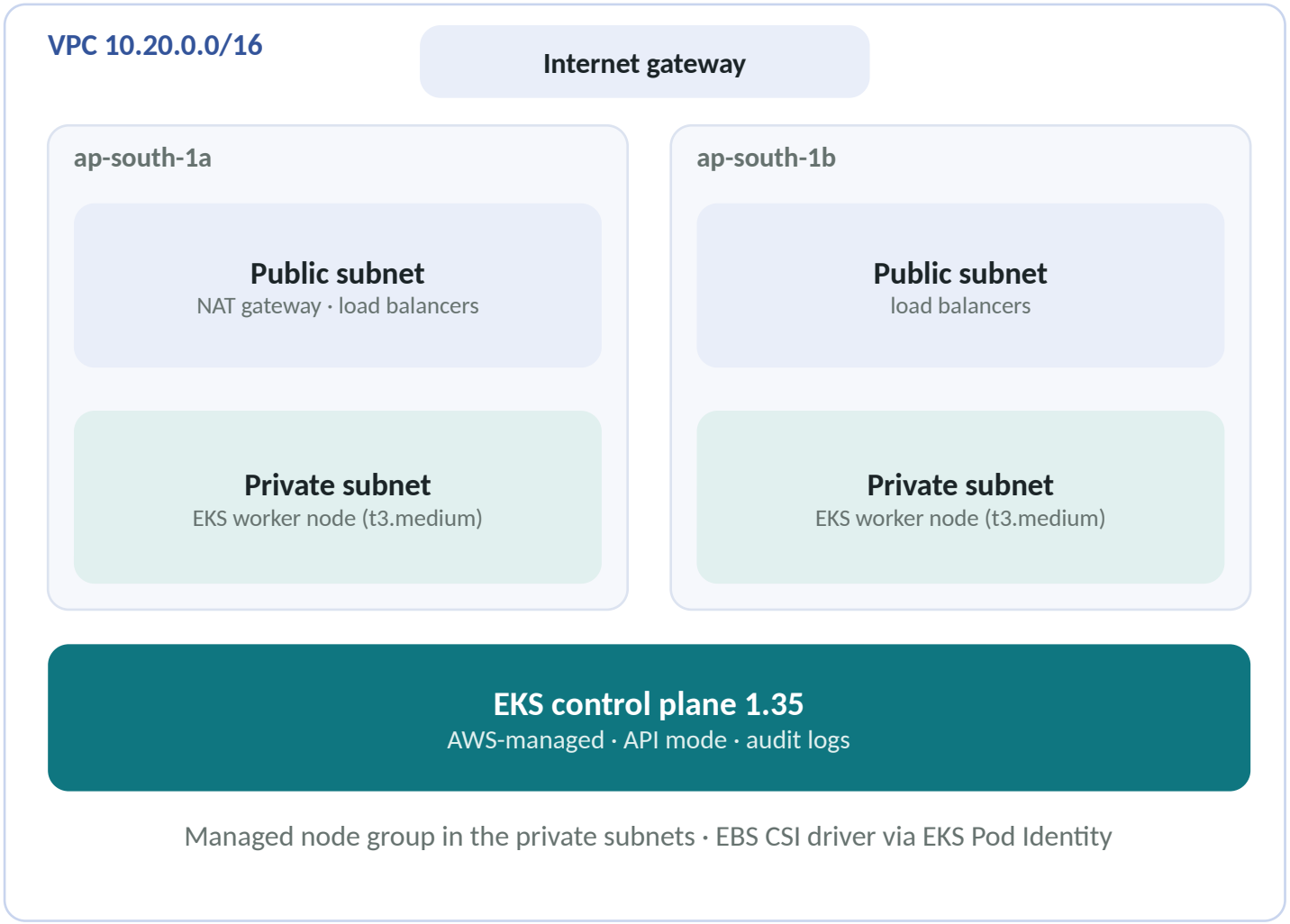
What the Trivy gate lets through, and why

The gate fails a build on any HIGH or CRITICAL CVE that has a fixed version, and on any HIGH or CRITICAL secret finding in an image layer. Both images pass: nothing fixable, no secrets at those levels.

Scanned without --ignore-unfixed, the backend's Debian 13 base still shows 8 HIGH CVEs (0 CRITICAL) with no fix released. Example: CVE-2025-69720, a stack overflow in ncurses' infocmp (CVSS 7.8, local). Fixed upstream in 6.5-20251213; Debian's 6.5+20250216-2 is still marked affected.

They sit in OS packages the Python API doesn't call (ncurses, util-linux, systemd libraries), in a container that runs non-root with a read-only root filesystem. Blocking on them would stop every release with nothing to upgrade to; the digest-pinned base image is bumped when Debian ships fixes, and CI rescans every build.

AWS with Terraform: VPC and EKS in ap-south-1



28

resources in one plan, all plain HCL

12m 23s

terraform apply, end to end

< $1

for the whole apply → screenshots → destroy cycle

Kubernetes with Helm, deployed by Argo CD

Deployments

backend (HPA 2–6), frontend (2)

Services

ClusterIP for API, UI; headless for DB

Ingress

Traefik: / → UI, /api → API

StatefulSet + PVC

PostgreSQL 17

HPA · PDB · NetworkPolicy

scale, survive drains, isolate DB

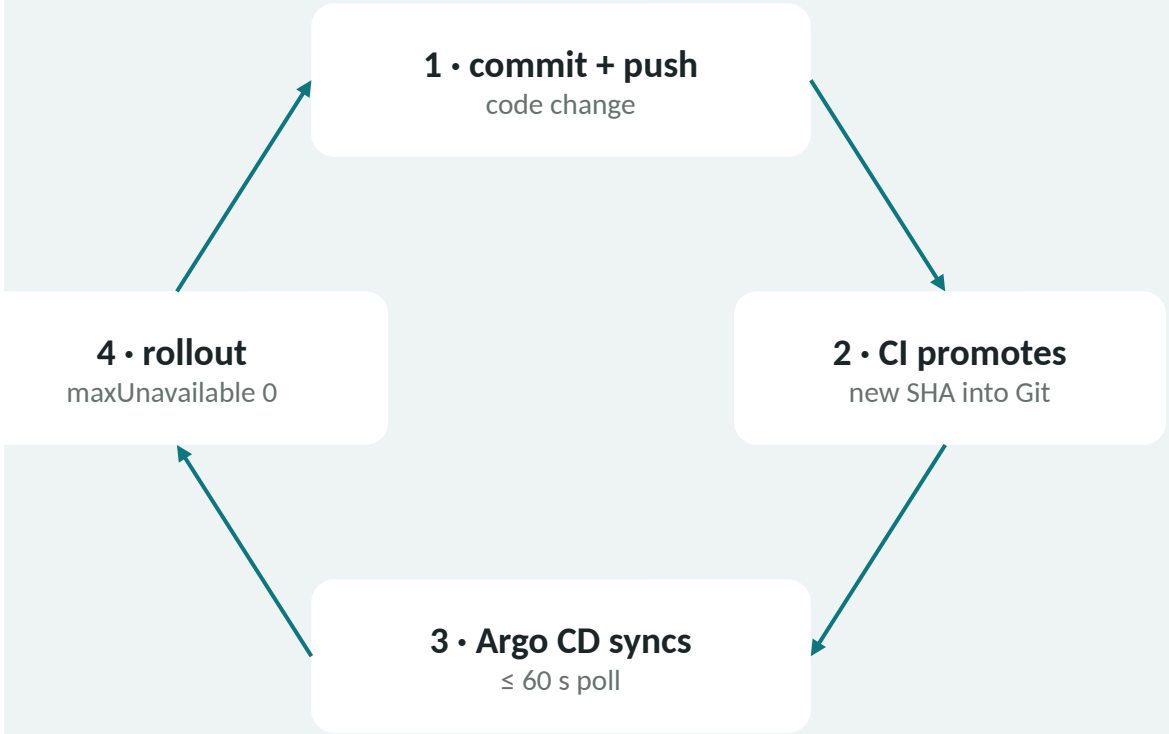
CronJob

lapse uncollected holds

ServiceMonitor · PrometheusRule

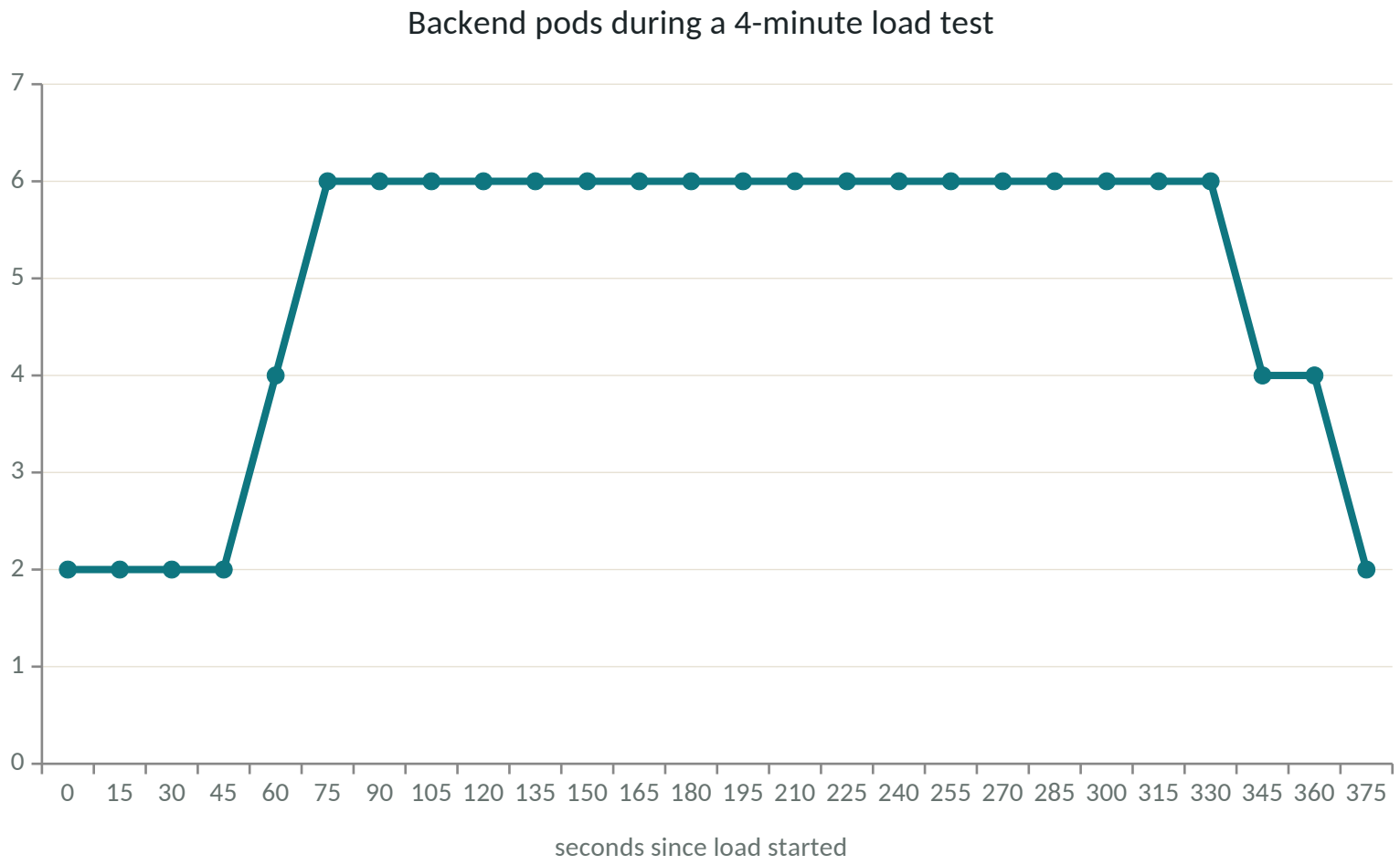
scrape + alert

The GitOps loop



Live demo: commit to Synced in 4 min 25 s (CI 3 min 35 s, then Argo CD picked up the promotion 48 s later)

Autoscaling under load



6

pods at peak (max 6)

45 s

until the first scale-out decision (all 6 pods by 75 s)

2m 20s

after load stopped, back to 2

Observability: Prometheus, Grafana and alerts

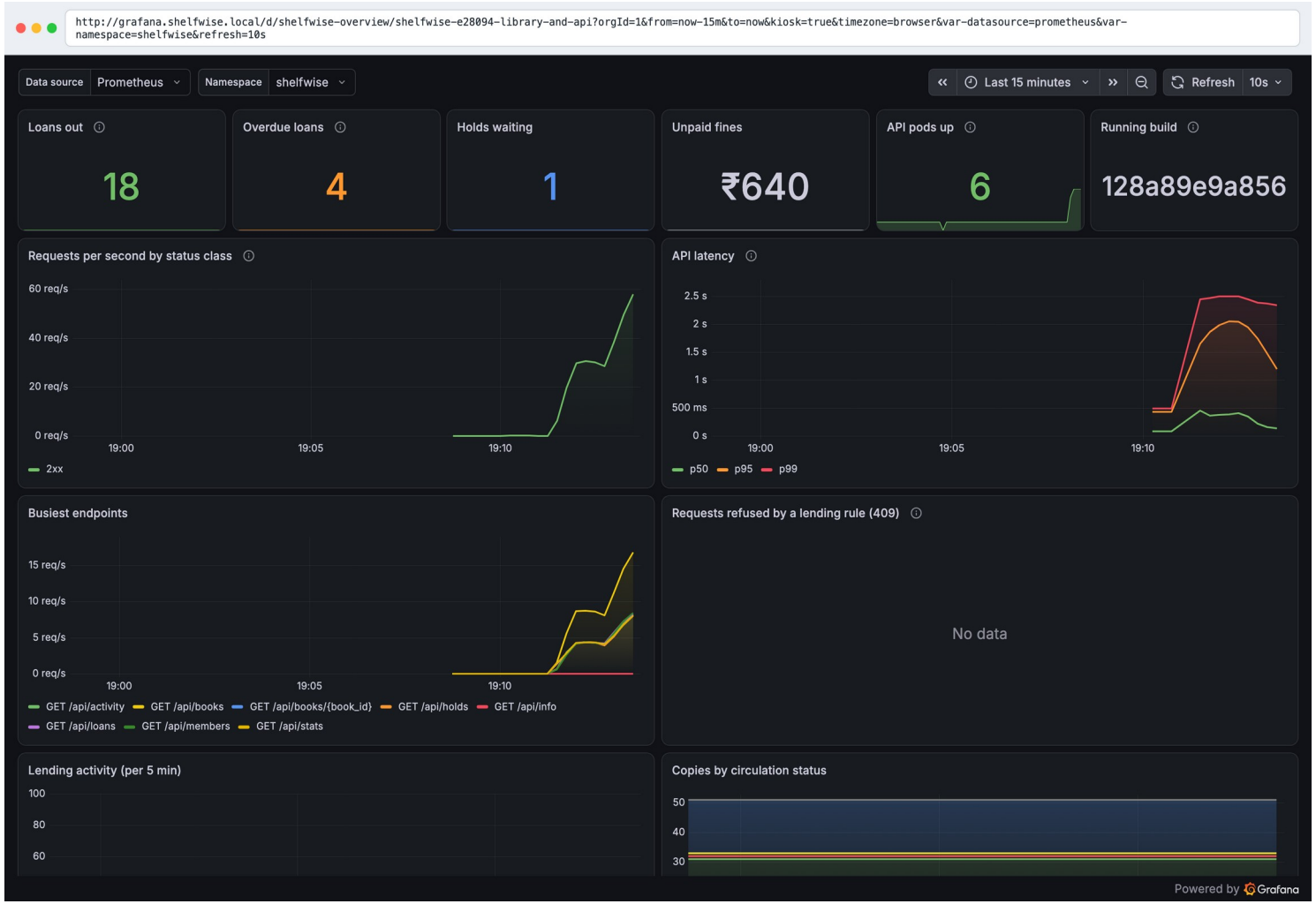
What the API exposes at /metrics

- HTTP rate, latency and status per route (buckets from 5 ms)
- Loans out, overdue, holds, unpaid fines, copies by status — read from the DB at scrape time
- Refusals per rule (409s) and the running build SHA

Alerts

- API down · 5xx > 5% · p95 > 500 ms
- Database unreadable
- Business: too many overdue loans

6 / 6 API pods scraped and UP at peak load



Key learnings

Correctness lives in the database

Row locks in a fixed order, backed by constraints. Thread-based race tests lied; process-based ones proved it.

Ship exactly what you scanned

CI saves the scanned image as a tarball and pushes that, never a rebuild.

Git is the deploy button

CI has no cluster credentials. A promotion commit is the release, and Argo CD does the rest.

Defaults hide problems

Prometheus ignores other releases' ServiceMonitors; latency buckets start at 100 ms; Trivy ignores misplaced options.

Pin everything

Actions by commit SHA after the 2026 Trivy-action tag hijack; base images by digest; charts by version.

Retired tools are a security risk

ingress-nginx was archived in March 2026, so this cluster uses Traefik with standard Ingress objects.

Limitations and what I would do next

PostgreSQL runs in the cluster	Amazon RDS with backups
One NAT gateway	One per AZ
No login on the API	OIDC
Secrets via kubectl	External Secrets
Local TLS off	cert-manager
GitOps on kind	Argo CD on EKS

Thank you

Questions, and the live walkthrough in the demo video

github.com/shubhamkumar9199/devops-heros/tree/main/session21-python

Green run: github.com/shubhamkumar9199/devops-heros/actions/runs/38908055488